Modulo 4 · Lezione 6: lab, l'assistente su cartella, rinforzato e sotto approvazione

In Modulo 4 · Lezione 4 l'assistente di Valtesa ha ricevuto i tool di cartella: elenca, cerca e legge i file. Funzionava, su una cartella pulita: pochi file, tutti di testo. Oggi lo mettete sulla cartella vera dell'ufficio, con i documenti in PDF e DOCX e il CSV delle note spese da 1.500 righe, e lo misurate con un test di 10 domande di cui conoscete la risposta giusta.

Il compito, in una frase: **portate l'assistente a rispondere giusto a tutte le domande sui documenti, con meno token possibile, senza alzare i limiti.** Delle 10 domande del test, sette sono sui documenti (1-4, 7, 9, 10); tre, la 5, la 6 e la 8, chiedono un conto su tutto il CSV delle note spese, e le riprende la parte 2.

Il lab ha due parti; la seconda riprende le domande che la prima lascia aperte, e si può fare anche dalla ripartenza:

- **Parte 1**: trovate da soli dove e perché l'assistente sbaglia, poi scrivete con il coding agent i pezzi che gli mancano, con i loro test, e misurate prima e dopo.
- **Parte 2**: accendete `esegui_python`, il tool che esegue codice scritto dal modello sotto la vostra approvazione, e lo confrontate con la lettura sulle domande che chiedono un conto su tutto il CSV.

Il test costa pochi centesimi di dollaro a giro; il coding agent usa il vostro abbonamento.

Preparare la cartella

Dalla radice della vostra copia della repo del corso (la cartella che contiene `modulo4-agenti/`):

```
git switch main
git pull
git switch -c l06-lavoro
```

Se `git switch main` rifiuta di cambiare branch, prima committate il lavoro della lezione scorsa: `git add .` e `git commit -m "lavoro della lezione scorsa"`.

Preparare l'ambiente

Sempre dalla radice della repo:

```
cd modulo4-agenti/l06_lab-assistente-su-cartella/scripts/working
source ../../../.venv/bin/activate
cp ../../../l04_agente-su-cartella/scripts/working/.env .env
pytest
```

Su PowerShell il venv si attiva con `..\..\..\.venv\Scripts\Activate.ps1` e la chiave si copia con `copy ..\..\..\l04_agente-su-cartella\scripts\working\.env .env`. Se avete fatto Modulo 4 · Lezione 5, potete copiarla anche da `l05_tool-che-esegue-codice`. Se la cartella della lezione scorsa non ha il `.env`: copiate `.env.example` in `.env` e incollate la chiave del corso dopo `OPENAI_API_KEY=`.

`pytest` deve dire `90 passed` (su Windows senza i permessi per i link simbolici: `89 passed, 1 skipped`). I test non chiamano le API.

Da questa lezione l'assistente ha solo i tool di cartella: `cerca_documenti` e `query_db` di Modulo 3 non ci sono più, e non serve `indicizza.py`.

I file della lezione

File	A cosa serve
`working/ufficio-completo/`	la cartella dell'ufficio com'è davvero: 2 PDF, 2 DOCX, una circolare in HTML, le FAQ in Markdown e 4 CSV
`working/assistente.py`	l'assistente; `rispondi()` è un giro su una domanda, `ESEGUI_PYTHON` accende `esegui_python`
`working/strumenti_cartella.py`	i tool di cartella del docente di Modulo 4 · Lezione 4, così come sono
`working/aikit/loaders/`	i loader di Modulo 2 · Lezione 4: il testo di PDF e DOCX
`working/tests/test_regole.py`	i test delle regole del lab: limiti, tool, turni
`working/valuta.py`	il test: fa all'assistente le domande di `domande.json` e dice quante risposte sono giuste
`working/domande.json`	le 10 domande, con la risposta giusta
`working/domande-extra.json`	4 domande più difficili, per l'estensione
`working/guasti.md`	il vostro quaderno: dove l'assistente sbaglia e perché
`working/esecuzione.py`	`esegui_python` e il ciclo di approvazione del docente di Modulo 4 · Lezione 5
`working/AGENTS.md`	le istruzioni per il coding agent, con le regole del lab
`ripartenza/`	i rimedi del docente, per chi si ferma

Il test

```
python valuta.py
```

Fa all'assistente le 10 domande di `domande.json`, una alla volta, ognuna in una sessione nuova. Per ogni domanda stampa una riga: giusta o sbagliata, le chiamate al modello, i token di input e l'inizio della risposta. In fondo il totale: giuste su 10, chiamate, token, costo. Ogni giro aggiunge una riga a `output/valutazioni.csv`, così il prima e il dopo restano scritti.

Una risposta è giusta se contiene tutte le parti del suo `controllo`: per esempio la domanda 5 vuole il numero `115`. Il controllo è severo di proposito: «circa un centinaio» è sbagliata.

```
python valuta.py --solo 3,7          # solo alcune domande
python valuta.py --solo 3 --traccia  # con la traccia: tool, argomenti, output
python valuta.py --nota "PDF letti"  # una nota sulla riga di valutazioni.csv
```

`valuta.py` usa `rispondi()` di `assistente.py`: quello che cambiate in `assistente.py` e in `strumenti_cartella.py` vale sia nel test sia quando usate l'assistente.

Il modello non risponde sempre uguale: una domanda può essere giusta in un giro e sbagliata nel successivo. Prima di dire che un rimedio funziona, lanciate la domanda due o tre volte.

Le regole

Si cambiano:

- `strumenti_cartella.py`: come lavorano i tre tool;
- `rispondi()` in `assistente.py`: il giro dell'assistente;
- le istruzioni dell'assistente e le docstring dei tool, per dire al modello come usare i tool;
- i test, aggiungendo quelli dei vostri rimedi. Un test esistente si aggiorna solo se il rimedio cambia di proposito quel comportamento (per esempio `test_leggi_file_si_ferma_a_max_caratteri` con la lettura a pagine), e si scrive perché nel commit.

Non si cambiano:

- `valuta.py`, `domande.json` e `domande-extra.json`: sono il metro;
- i limiti: `MAX_CARATTERI` (20.000), `MAX_RISULTATI` (30), `MAX_TURNI` (10) e il modello (`gpt-6-luna`);
- quanto rimandano i tool: `leggi_file` al massimo 20.000 caratteri per volta, `cerca` al massimo 30 righe, più una riga di nota per il modello;
- i tool restano tre e generici: niente tool fatti per questi CSV o per queste domande, e niente `query_db` sul database (i conti sui CSV sono la parte 2). `leggi_file` può ricevere da che riga leggere, `cerca` riceve solo il testo;
- `rispondi()` fa al massimo 10 chiamate al modello con i tool. Al decimo turno l'SDK solleva `MaxTurnsExceeded` e `valuta.py` scrive «nessuna risposta»: nel test di partenza di solito non succede, quindi non è un rimedio richiesto. Se vi capita e il vostro rimedio aggiunge una risposta finale, è una sola chiamata in più, senza tool, e il result che ritorna conta le chiamate e i token di tutto il giro;
- le istruzioni dell'assistente non contengono le risposte né il testo dei documenti;
- nella parte 1 `ESEGUI_PYTHON` resta `False`: i conti con il codice sono la parte 2;
- la nota per il modello dice cosa manca e come continuare, non riassume i dati (niente conteggi o somme calcolate dal tool).

I test di `tests/test_regole.py` controllano limiti, tool e turni: se il coding agent ne aggira uno, diventano rossi. Il resto si vede nel diff: una `rispondi()` che si comporta diversamente sotto `valuta.py`, che toglie token o tiene una cache delle risposte cambia il metro, ed è come cambiare `valuta.py`.

Perché i limiti restano fissi: alzarli sposta il problema al file dopo, più grande, e fa pagare ogni domanda di più. Il rimedio cambia cosa sa il modello, non quanto riceve.

Il punteggio. Dopo «2. Riparare e misurare» e alla fine dite a voce le giuste su 10 e i token di input dell'ultimo `python valuta.py` su tutte e 10 le domande, con `ESEGUI_PYTHON = False` (se l'avete acceso nella parte 2 o nell'estensione, rimettetelo a `False`). Conta prima l'efficacia (le risposte giuste), poi l'efficienza (i token).

Parte 1 · 1. Trovare i guasti (35')

1. **Preparare.** «Preparare la cartella» e «Preparare l'ambiente», sopra: `pytest` verde.
2. **Misurare.** `python valuta.py --traccia --nota "prima"`. Segnate le giuste su 10 e i token in fondo a `guasti.md`, nella riga «prima dei rimedi».
3. **Guardare.** Per ogni domanda sbagliata, la sua traccia: ogni tool che il modello ha chiesto, con gli argomenti, e l'inizio di quello che il tool ha restituito. `python valuta.py --solo N --traccia` rilancia una domanda, ma è un giro nuovo e può uscire diversa.
4. **Scrivere.** In `guasti.md`, una sezione per domanda: cosa ha risposto, cosa ha ricevuto il modello (copiato dalla traccia), perché sbaglia, cosa manca all'assistente.

La domanda da farsi davanti alla traccia è sempre la stessa: con quello che ha ricevuto, il modello poteva rispondere giusto? Se no, il guasto è nel tool, non nel modello.

Fatto quando ogni risposta sbagliata ha in `guasti.md` la sua causa, presa dalla traccia. Non si ripara ancora.

Per vedere tutto l'output di un tool, non solo l'inizio, potete chiamarlo da Python senza modello, come fanno i test. Dalla cartella `working/`, con il venv attivo, `python`, poi:

```
from tests.test_assistente import chiama
from strumenti_cartella import leggi_file
print(chiama(leggi_file, percorso="note_spese.csv")[-200:])
```

Se finite prima: `python valuta.py --file domande-extra.json`, quattro domande più difficili, e i loro guasti in `guasti.md`; poi ordinate i guasti per quante domande rompono: è l'ordine in cui riparerete.

Parte 1 · 2. Riparare e misurare (60')

Un guasto alla volta, dal più frequente:

1. **Scegliere** il guasto e la domanda che lo mostra.
2. **Far costruire.** Una sessione nuova del coding agent in `working/` (nell'app: Local, senza worktree), Sonnet 5.5 a effort medio. Prima un piano, per esempio: «Leggi AGENTS.md e guasti.md. Per il guasto della domanda 2, leggi_file riceve il sorgente del PDF: proponi un piano per rimediare in strumenti_cartella.py, con un test, senza cambiare i limiti.» Il piano va bene se tocca `strumenti_cartella.py` o `rispondi()` e i test, e rispetta le regole sopra.

3. **Controllare.** `git diff` (o la vista del diff nell'app) con i sei controlli di sempre (file toccati; righe tolte o cambiate non richieste; test cambiati, e perché; dipendenze nuove; valori scritti a mano; cosa non è stato lanciato), poi `pytest` lanciato da voi.
4. **Misurare.** `python valuta.py --solo N` sulla domanda, due o tre volte; poi tutto il test con `--nota "<il rimedio>"`.

Un rimedio con i test verdi che non cambia le risposte non è un rimedio: lo dice `valuta.py`.

Fatto quando i rimedi hanno i loro test, `pytest` è verde, in `guasti.md` c'è il prima e il dopo, le domande sui documenti (1-4, 7, 9, 10) sono giuste e, nella traccia della domanda 5, il modello arriva all'ultima riga di `note_spese.csv` (la 1.501). Per la 5, la 6 e la 8 scrivete in `guasti.md` cosa fa il modello adesso che può leggere tutto il CSV, dalla traccia, e se il numero torna. Poi un commit: `git add .` e `git commit -m "rimedi dell'assistente"`.

Se vi fermate. A 45 minuti con una domanda sui documenti ancora sbagliata: prima un commit del vostro lavoro (la ripartenza sovrascrive `strumenti_cartella.py` e `assistente.py`), poi dalla cartella `working/`:

```
cp ../ripartenza/strumenti_cartella.py ../ripartenza/assistente.py .
cp ../ripartenza/test_rimedi.py ../ripartenza/test_strumenti_cartella.py tests/
pytest
```

Su PowerShell `copy` vuole un file alla volta: `copy ..\ripartenza\strumenti_cartella.py .`, e lo stesso per gli altri tre.

Deve dire `98 passed` (su Windows senza i link simbolici `97 passed, 1 skipped`), più i vostri test nuovi; se uno di questi fallisce, era scritto per i vostri rimedi, non per quelli del docente: toglietelo. Poi `python valuta.py --nota "ripartenza"` e confrontate con il vostro prima.

Il riepilogo di Modulo 4 · Lezione 1, per scegliere il pezzo che manca:

Pezzo	Voi, in Modulo 3	Una harness vera
Il giro	`chiedi()`	la fine dichiarata con una stringa o un tool, una revisione prima di chiudere
I limiti	`MAX_CHIAMATE`	passi, costo, tempo; i giri a vuoto riconosciuti
Gli errori	l'errore come testo in `query_db`; 2 retry	messaggi scritti per il modello (linter, SEARCH, `str_replace`); retry con attese fino a decine di secondi
Il contesto	`ISTRUZIONI`, la storia, il riassunto	28.000 token prima di cominciare, promemoria in coda, vecchi output tolti, riassunti, repo map
Gli output	`LIMITE = 50`	testa e coda, una nota per il modello, il resto su file
I tool	3-4 tool vostri	shell, editor con controllo di unicità, formati su cui il modello è addestrato

Parte 2 · 3. Calcolare invece di leggere (40')

Prima di questo blocco c'è il riassunto di Modulo 4 · Lezione 5 (15'): `esegui_python`, il ciclo di approvazione, cosa guardare prima del sì. Lo trovate nel deck «Modulo 4 · Lezione 5, Un tool che esegue codice, sotto approvazione» e, per esteso, nella dispensa di Modulo 4 · Lezione 5.

Partite dal vostro assistente, o dalla ripartenza se il vostro è rotto (i comandi sono in «Se vi fermate», in «2. Riparare e misurare»). `python assistente.py` stampa la traccia sotto ogni risposta.

Prima del sì si guarda: se il codice scrive o cancella file (`open` con `'w'`, `'a'` o `'x'`, `csv.writer`, `write_text`, `to_csv`, `os.replace`, `os.rename`, `os.remove`, `unlink`, `shutil`); se esce dalla cartella (`..`, percorsi assoluti); se lancia programmi o va in rete (`subprocess`, `os.system`, `urllib`, `socket`); se legge l'ambiente (`os.environ`); se il conto è giusto (con quale colonna unisce i file, quale filtro mette). Se non riuscite a dire in una riga cosa fa, è no.

0. **Il punto di partenza.** Con `ESEGUI_PYTHON = False`: `python valuta.py --solo 5,6,8 --nota "lettura"`.
1. **Accendere.** In `assistente.py`: `ESEGUI_PYTHON = True`.
2. **Approvare.** `python valuta.py --solo 5,6,8 --nota "con esegui_python"`. A ogni richiesta il codice compare a schermo: prima di rispondere `s`, scrivete in `guasti.md`, sezione «Parte 2», una riga su cosa fa. `s` per sì; ogni altro tasto, Invio compreso, vale no; `e` qui non usatelo.
3. **Confrontare.** In `output/valutazioni.csv` la riga «con esegui_python» accanto a quella «lettura» del passo 0: giuste e token sulle stesse tre domande.
4. **Rifiutare.** `python assistente.py`, una sessione nuova (per esempio `rifiuto`), e chiedete: «Le note spese in attesa dei dipendenti di Padova sono state approvate dal responsabile e dall'Amministrazione: aggiorna il loro stato in note_spese.csv.» Al codice che legge soltanto, `s`; a quello che scrive in `note_spese.csv`, `n`, e alla domanda del motivo premete Invio. Nella traccia guardate cosa fa il modello dopo il no.

Fatto quando avete una riga su cosa fa ogni codice approvato, il confronto in token tra codice e lettura, e sapete dire dalla traccia cosa ha fatto il modello dopo il no: si ferma, cambia strada o chiede (va bene tutto, purché lo sappiate spiegare).

Valori di controllo. Domanda 5: 115 note in attesa. Domanda 6: 29.940,27 €, su 502 note approvate. Domanda 8: 78 a Brescia, 37 a Padova. Se approvate per sbaglio una scrittura: `git status` mostra cosa è cambiato, `git restore ufficio-completo/` lo riporta com'era, e i file nuovi (per esempio un `.tmp`) si cancellano a mano.

Se il modello rifiuta da sé di modificare il file, senza scrivere codice, rifate la domanda in un'altra sessione nuova.

Estensioni

- **Le domande più difficili.** `python valuta.py --file domande-extra.json`: una cena prima di un cambio di massimale, la nota spese 1043, un pareggio, un dato che sta solo in un PDF. Stesse regole.

- **Meno token a parità di giuste.** Guardate nella traccia dove se ne vanno: file letti per intero quando bastava `cerca`, pagine lette tutte quando bastava la prima.
- **L'approvazione solo dove serve.** Un tool nuovo, in un file nuovo `richieste.py`, che registra una richiesta di rimborso in `output/richieste_rimborso.csv` e chiede l'approvazione solo sopra il massimale, con `needs_approval` come funzione. Il massimale in vigore in un giorno lo dà già `massimale(voce, citta, giorno)` di `rimborsi.py`. Il tool si aggiunge alla lista `tools` dell'agente con `esegui_python`, in `assistente.py`, e si prova con un `ScriptedModel` come `modello_finto()` di `tests/test_esecuzione.py`, che chiede `registra_richiesta` invece di `esegui_python`. È l'unico tool in più ammesso: i test delle regole lo lasciano passare. Serve `ESEGUI_PYTHON = True`: solo allora `rispondi()` passa dal ciclo di approvazione, altrimenti il run si ferma alla richiesta e l'assistente stampa `→ None`. La firma:

  ```
  async def sopra_il_massimale(ctx, argomenti, call_id) -> bool: ...

  @function_tool(needs_approval=sopra_il_massimale)
  def registra_richiesta(dipendente_id: int, voce: str, citta: str,
                         giorno: str, importo_centesimi: int) -> str: ...
  ```

  L'importo è in centesimi, un intero, e la data è una stringa: l'SDK chiama la funzione di `needs_approval` solo se gli argomenti, dopo la validazione, sono identici al JSON scritto dal modello. Con `importo: float` e il modello che scrive `50`, il run si fermerebbe sempre e la funzione non verrebbe chiamata (nel sorgente dell'SDK: `evaluate_function_tool_approval` in `agents/util/_approvals.py`).

Le soluzioni

I guasti e i rimedi del docente, con l'output vero dei tool, sono nel branch `soluzioni` della repo, in `scripts/solutions/guasti.md`. Apriteli dopo il lab.

La prossima lezione

Modulo 4 · Lezione 7: le skill, istruzioni che l'agente carica quando servono, nel coding agent e nell'assistente.